6ELEN018W Applied Robotics - Assignment Report

Fridalí Gaias w1905050

January 1, 2026

1 Introduction

The objective of this coursework is to design and implement intelligent control algorithms for a mobile robot operating in an unknown grid based environment (Dracopoulos, 2025). The robot's goal is to navigate from an arbitrary starting position (I) to a target goal (G) while avoiding obstacles. Two distinct approaches were implemented and evaluated:

1. **Reinforcement Learning (RL)**: A Q-Learning algorithm implemented from scratch to learn optimal policies through interaction with the environment.
2. **Neural Network (NN)**: A Multi-Layer Perceptron (MLP) trained using Supervised Learning to predict optimal actions based on the robot's coordinates.

This report details the implementation, parameter tuning, and comparative performance of these two controllers, specifically analyzing their ability to handle complex obstacle configurations.

2 Reinforcement Learning Controller (Tasks 1 & 3)

2.1 Algorithm Implementation

The Q-Learning algorithm was implemented without any external libraries, as per coursework specifications, extending the basic concepts provided in the module materials (Dracopoulos, 2025). The module code provided a basic Q-learning structure for a fixed graph. I extended this to a dynamic `MazeEnvironment` class using `numpy` arrays to represent the 6×6 grid. The state space was adapted from symbolic dictionary keys to numerical coordinate tuples `(row, col)`.

The environment coordinates were derived from the visual representation in Figure 1: the initial state, I, visually located at row 4, column 3, was converted to Python's 0-based indexing as (3, 2), the goal position, G, visually located at row 2, column 6, was converted to Python's 0-based indexing as (1, 5). The step function was extended to include boundary checks and collision detection. If a move leads to an obstacle `(grid[r,c] == 1)`, the robot remains in the current state, simulating a collision.

The core mechanism updates a Q-table $Q(s, a)$ representing the expected utility of taking action a in state s. The update rule applied is:

$$Q(s, a) = r + \gamma \ \cdot \ \max Q(s', a')$$

where r is the immediate reward and γ is the discount factor.

My reward structure:

- **Goal state**: a reward of `+100` was assigned when the robot reached the goal.
- **Step/Obstacle**: a reward of `0` was assigned for all other transitions.
- I chose this sparse reward structure, combined with a discount factor of $\gamma = 0.9$, to encourage the shortest path. Since rewards at the goal are discounted by 0.9 for every step taken, a longer path results in a lower Q-value at the start state, making the agent be more efficient.

```
Maze initialised. Number of free cells: 28
Task 1 config : start at I: (3, 2), goal at G: (1, 5)
Generating CLEAN dataset for 28 possible goal positions . . .
Dataset generation is done. Total samples: 756

--- Solving I ---> G ---
Tracing path from (3, 2) to (1, 5) . . .
Path taken: [(3, 2), (4, 2), (4, 3), (5, 3), (5, 4), (5, 5), (4, 5), (3, 5), (2, 5), (1, 5)]
Result: Success! Goal has been reached !!
 X Shape: (756, 4), y Shape: (756,)
```

Figure 1: Q-Learning controller output for the maze problem; successful.

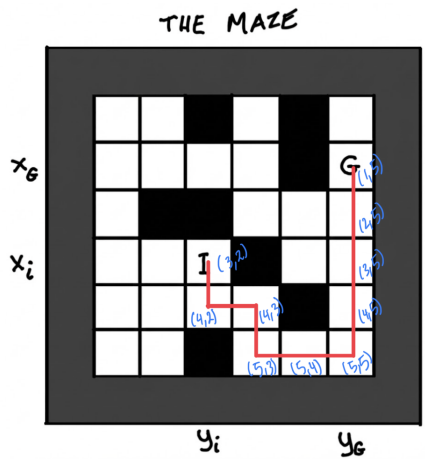


Figure 2: The most optimal path, taken by agent.

2.2 Results (Task 1)

The Q-Learning controller successfully converged on the optimal path.

- **Start**: (3, 2)
- **Goal**: (1, 5)
- **Path Taken**: [(3, 2), (4, 2), (4, 3), (5, 3), (5, 4), (5, 5), (4, 5), (3, 5), (2, 5), (1, 5)]
- The path length is 9 steps. The agent successfully navigated around the local obstacles demonstrating that the Q-table correctly propagated the goal reward back to the start state.

2.3 Task 3: File-Based Maze Solving

To demonstrate adaptability, the system was extended to parse maze configurations from text (.txt) files in the given structure of the example from the assignment specifications. My implementation correctly interprets then parses dimensions, start/goal coordinates, and the obstacle grid.

The Q-Learning controller successfully navigates the file-generated mazes, proving the deterministic nature of the algorithm allows it to solve any static problem.

```
Maze loaded successfully!
Maze dimensions: 6x5
Start position: (0, 1)
Goal position: (2, 4)
Running Q-Learning on File Maze . . .
Tracing path from (0, 1) to (2, 4). . .
Path: [(0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4)]
Success! The maze provided by the txt file has been solved!
```

Figure 3: Output for File-Based problem 1, '*maze_ config.txt*'; successful. The algorithm was then successfully tested on further maze configurations to demonstrate robustness (see Appendix).

3 Neural Network Controller (Task 2)

3.1 Initial Development and Challenges

I implemented the Neural Network using an `MLPClassifier` from the Scikit-learn library (Scikit-learn Developers, 2024) to predict the optimal action (N, E, S, W) given the robot's current position and goal coordinates. The `MLPClassifier` was chosen because the task involves selecting one of four discrete actions, not predicting a continuous value (unlike the example from the module material (Dracopoulos, 2025). A regressor would wrongly treat the directions as ordered numerical values, while a classifier is better suited to choosing between distinct actions.

The initial model used a standard architecture with two hidden layers of 20 neurons each `hidden_layer_sizes=(20, 20)` and limited training `(max_iter=1000)`, parameters derived from the module material (Dracopoulos, 2025) .

Initial Performance: Early testing yielded poor results, with a model accuracy of approximately 73.68%. Crucially, when testing the specific path from $I(3, 2)$ to $G(1, 5)$, the controller failed completely, the robot just looped at the start position $(3, 2)$.

The network correctly identified the general goal direction (North-East) but seemed to fail to account for the immediate obstacles.

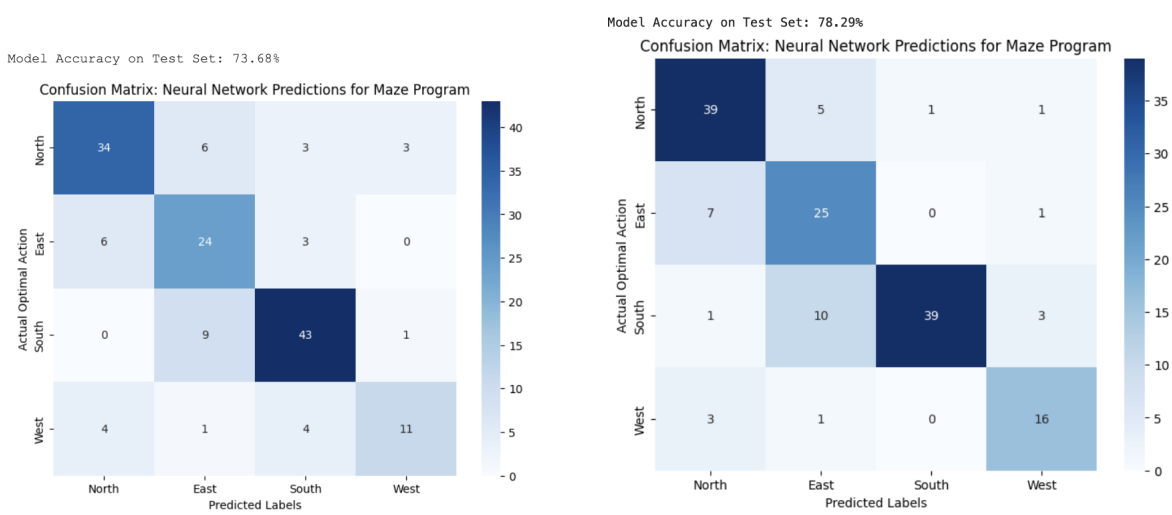
3.2 Optimisation and Parameter Tuning

To address the navigation failure, I applied specific optimisations to the code:

1. Added an IF statement in genereate_full_dataset in the hopes of improving filtering as the robot gets stuck:
   `if np.max(Q_table[sx, sy, :])  != 0:`
2. The network capacity was increased to handle the problem, as the initial values had been taken from the tutorial (Dracopoulos, 2025) example. The initialisation was updated to:
   ```
   mlp = MLPClassifier(
   hidden_layer_sizes=(100, 100), # Increased from (20,20)
   max_iter=5000, # Increased from 1000
   )
   ```

3.3 Final Evaluation

Following the aforementioned optimisations, the model's accuracy on the test set improved from 73.68% to 78.29%. The NN achieved a Training Accuracy of 82.95% and a Test Accuracy of 78.29% on unseen data. This small performance gap (less than 5%) shows that the model did successfully generalise without overfitting.



(a) Initial Test.

(b) After optimisation.

Figure 4: Neural Network Accuracy and CM through testing.

Classification Report:

	precision	recall	f1-score	support
North	0.77	0.74	0.76	46
East	0.60	0.73	0.66	33
South	0.81	0.81	0.81	53
West	0.73	0.55	0.63	20
accuracy			0.74	152
macro avg	0.73	0.71	0.71	152
weighted avg	0.74	0.74	0.74	152

(a) Initial Test.

Classification Report:

	precision	recall	f1-score	support
North	0.78	0.85	0.81	46
East	0.61	0.76	0.68	33
South	0.97	0.74	0.84	53
West	0.76	0.80	0.78	20
accuracy			0.78	152
macro avg	0.78	0.79	0.78	152
weighted avg	0.81	0.78	0.79	152

(b) After optimisation.

Figure 5: Neural Network Classification Reports through testing.

The final confusion matrix in Figure 2 highlights a persistent issue: the model frequently seems to confuse North and East; specifically, when the optimal action was East, the model incorrectly predicted North 7 times. Furthermore, the classification report reveals that while the South action had very high precision (0.97), the East action had the lowest precision (0.61). This also indicates the model is unreliable when predicting East movements, often mistaking them for North.

Despite the accuracy improvement, the robot still failed to complete the specific test path from $I(3,2)$ to $G(1,5)$. The output shows that the robot remains at coordinate `(3,2)` for the duration of the test.

```
--- Testing Neural Network Robot on I ---> G Path ---
Neural Network Path from (3, 2) to (1, 5) . . .
Path taken by NN (neural network): [(3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2)
    , (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3, 2), (3,
    2), (3, 2)]
Failure ... Unfortunately, the Neural Network has failed to reach the goal...
```

Figure 6: NN output for the maze problem; unsuccessful.

It seems that the program cannot find a solution past the obstacle, the model (as shown by the reports) seems to be biased towards predicting "North", since North is blocked, the robot keeps colliding with the obstacle and remains at `(3,2)`.

This confirms that while NNs generalise well, they lack the specific state awareness required to navigate obstacle traps, definitely in this problem.

4 Conclusion

My coursework contains successful implementation of the two control paradigms; the Reinforcement Learning approach proved superior for this static environment/problem, achieving a 100% success rate as it was able to completely map the state space, and reach the Goal in the most efficient way possible, both in the initial default maze given and the third task where the model was generalised for a maze formed by a .txt file. The Neural Network, while successfully optimised up to 78.29% accuracy (an improvement from the initial tests), it ultimately failed to navigate the problem scenario. This failure highlights that while Neural Networks offer generalisation, they lack the precision of deterministic algorithms like Q-Learning when operating on data.

However, it is important to recognise that the main strength of NN lies in its predictive ability and its potential to generalise. Unlike the Q-learning approach, which requires every possible state to be visited and stored (which means it quickly suffers from the issue of dimensionality in larger environments), A NN instead learns an underlying function that relates inputs to outputs, which allows it to generalise from what it has already learned and select reasonable actions even in states it has not explicitly encountered.

The poor performance of the NN can largely be explained by the discrete, grid-based nature of the environment and the use of Cartesian coordinates (x, y) as inputs. These coordinates offer very limited information, making it difficult for the network to learn meaningful generalisations. On the other hand, in real world robotic systems with continuous state spaces, such as autonomous driving, the ability to generalise from learned patterns is essential. In these settings, a Q-learning approach like the one used in Task 1 would be computationally impractical, while a Neural Network based approach is far more suitable.

5 References

Dracopoulos, D. (2025) 6ELEN018W Applied Robotics Coursework Specification 2025/2026. London, University of Westminster. (Accessed December 2025)

Dracopoulos, D. (2025) *'Lecture 8: Robot Control - Intelligent Control Algorithms'*, 6ELEN018W Applied Robotics. University of Westminster, unpublished. (Accessed December 2025)

Dracopoulos, D. (2025) *'Tutorial 9: Reinforcement Learning Exercises'*, 6ELEN018W Applied Robotics. University of Westminster, unpublished. (Accessed December 2025)

Scikit-learn Developers (2024) Neural Network Models (Supervised).
Available at: https://scikit-learn.org/stable/modules/neural_networks_supervised.html (Accessed: December 2025).

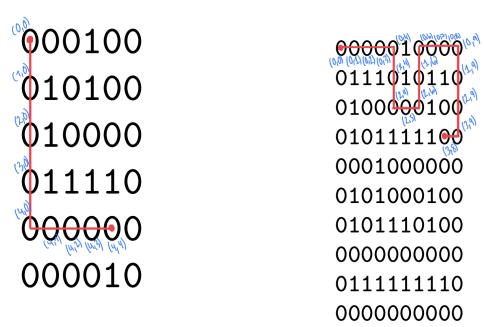
6 Appendix

```
Reading maze from maze_2.txt . . . please stand by ...
Maze loaded successfully!
Maze dimensions: 6x6
Start position: (0, 0)
Goal position: (4, 4)

Running Q-Learning on File Maze . . .
Tracing path from (0, 0) to (4, 4). . .
Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Success! The maze provided by the txt file has been solved!
```

Figure 7: Code execution output demonstrating my Q-learning algorithm successfully solving a second maze configuration (*'maze_2.txt'*) with different obstacles.

```
Reading maze from maze_3.txt . . . please stand by ...
Maze loaded successfully!
Maze dimensions: 10x10
Start position: (0, 0)
Goal position: (3, 8)

Running Q-Learning on File Maze . . .
Tracing path from (0, 0) to (3, 8). . .
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (2, 5), (2, 6), (1, 6), (0, 6), (0,
     7), (0, 8), (0, 9), (1, 9), (2, 9), (3, 9), (3, 8)]
Success! The maze provided by the txt file has been solved!
```

Figure 8: Execution results for a third maze configuration (*'maze_3.txt'*).



(a) *'maze_2.txt'* path. (b) *'maze_3.txt'* path.

Figure 9: Path visualisations of additional maze problems for Task 3.